

2026团队秋招算法题

大家好，欢迎各位新同学参与Dian团队秋招，加入我们的技术大家庭～

本题目面向对算法、AI方向感兴趣、擅长算法的大二同学，注意不要选错试题了。

题目

从神经网络基础出发，逐步完成 MLP、CNN 和 U-Net 的学习与实践，最终实现一个“输入带手写内容的试卷或作业图片，输出擦除手写内容后的干净图片”的完整项目。

每个 Level 都应保留代码、实验记录和 README，并使用 Git 管理版本。

要求

1. 建议使用 Python 开发，如果你非要使用 PTX Assembly/C/C++/Rust 咱也不拦着你；
2. 由于项目耗时较长，可能大家会忘记一些非常重要的细节，请你在学习过程中维护一个学习文档，记录你在工程开发中遇到的各种问题或思路
3. 建议大家养成良好的 git 历史记录与代码管理习惯。

提示

- 现在的大语言模型（LLM）发展迅速，甚至可以快速地完成整个题目。即便如此，我们还是希望你拥有坚实的基础，让“自己”学到有用的东西。因此我们设计了一个循序渐进的考察过程，希望你能在过关斩将的过程中收获成就感与知识。
- Vibe Coding 发展迅速，你一定听说过这个名词，甚至已经使用过一段时间。相比于编码，现在即便是最强的大模型在处理工程问题上还是有很多不足。能否发挥你作为人类的长处，驾驭各种 AI 呢？如果你有什么经验，不妨记录或者分享。
- 请不要吝啬与活人交流的机会。如果你遇到了什么问题，可以大胆在招新群提出来，说不定有随机刷新的同学、学长学姐解答；说不定其他人也遇到过相应的问题；当然，独立查找资料也是难能可贵的；
- **算法题不仅仅考察你的项目完成度如何，更看重你对不同神经网络与代码的深入理解。**

Level 0：环境管理

使用 Wsl 在 Windows 上使用 Linux 系统，下载 Miniconda 环境管理包，了解 Conda 的作用，体会不同项目使用的环境不同的意义与重要性。

推荐创建 `pytorch = 3.11.9` 的 Conda 环境完成后续任务，你可以使用类似这样的命令来判断你的 torch 是否正常 work 了。

代码块

```
1 import torch
2 print(torch.__version__) # PyTorch版本
3 print(torch.version.cuda) # CUDA版本
4 print(torch.cuda.is_available()) # True表示可用
```

你的电脑最好支持硬件加速，否则在完成后面任务的时候可能会耗时过久。如果本地无法运行，也许可以尝试租赁一个GPU服务器，或者尝试使用 Google Colab。

Level 1: 使用 MLP 完成 MNIST 手写数字识别

学习目标

从直觉、逻辑上理解 MLP 如何完成前向计算、计算损失，并通过反向传播更新参数，建立神经网络解决问题的基本思维，你可以通过思考一下的问题来加深自己的理解。

- 假设我们将MLP看成一个黑盒，只需要提供输入就可以得到一个输出作为我们的结果，那么我们对MLP的输入是什么样子的？图片是怎么被输入进MLP进行处理的？
- 一个MLP在没有经过任何的训练的情况下输入数据后应该会输出完全错误的结果，那么在直觉上，MLP是怎么通过训练让输出的结果越来越正确的？
- 如果我们将神经网络看作一个输入->输出的映射，那么要怎么理解基于输入和输出（基于数据）逆向映射的这一个过程？和对输入做变换得到输出有什么区别？

掌握 PyTorch 的基本训练流程，在AI的辅助下，并完成MNIST手写数字识别，你需要看懂AI的代码，理解数据的流动，前向计算和反向传播在代码里面是怎么进行的，如果你可以手写，那你会得到嘉奖(^v^)

- Tensor、Dataset、DataLoader 和自动求导
- nn.Module、优化器和训练/验证循环
- MNIST 数据格式、归一化和基础数据预处理
- 模型权重的保存、加载和推理

将 MNIST 图片展平后输入 MLP，训练手写数字分类模型，并编写脚本对单张图片进行推理。

验收标准

- 在不数据泄露的情况下，测试集准确率达到90%
- 能训练模型，并且推理模型，并且可视化、保存Loss曲线
- README 中记录网络结构、超参数和实验结果

Level 2：使用 CNN 完成 MNIST，并与 MLP 对比

学习目标

理解卷积神经网络处理图像的方式，以及它相比 MLP 的优势，你可以通过思考一下的问题来加深自己的理解。

- 有人认为：“CNN的本质是天然使用了图片的特征：相邻像素之间具有关联，因此相对 MLP CNN 具有更多的约束”我们要如何理解这一点？
- 理解不同超参数对模型性能的影响（你或许需要用上一个的观点去理解更大 or 更小卷积核对模型性能的影响）

在 AI 的辅助下，将 Level 1 的 MLP 替换为 CNN，通过对照实验理解不同网络结构的差异，观察性能是否有提升，如果有提升那么是为什么；如果没有提升那么是为什么。

实践任务

复用 Level 1 的数据和训练流程，训练 CNN 版本的 MNIST 分类器，并从准确率、参数量、收敛速度和错误样本四个角度与 MLP 对比。

验收标准

- CNN 测试集准确率建议达到 96% 左右
- 输出与 CNN 的训练曲线
- 能结合实验结果解释 CNN 更适合图像任务的原因

提示

如果你觉得 CNN 相较于 MLP 提升不明显，可以试试使用 Fashion-MNIST

Level 3：经典网络 AlexNet、VGG 与 U-Net

学习目标

理解经典 CNN 从图像分类到图像分割的发展，并为最终项目掌握 U-Net。

核心知识点

- AlexNet：深层 CNN、ReLU、Dropout 和数据增强
- VGG：重复堆叠小卷积核与网络模块化
- ResNet：残差跳跃使得模型更适合 Scaling
- U-Net：编码器、解码器、下采样、上采样和跳跃连接

实践任务

尝试在已有代码基础上修改，分别实现 AlexNet, ResNet

Level 4: 基于 U-Net 的手写笔记擦除

项目目标

基于Unet模型，在AI的辅助下完成手写笔记擦除任务，具体而言，我们需要输入一张带有手写答案、批注或演算内容的试卷/作业图片，输出尽量保留印刷文字、表格和题目结构，同时擦除手写内容的干净图片，数据集可见于https://pan.baidu.com/s/1HgP0fqd_V-VlR0j_oLAOEg?pwd=dian。

- 数据清洗、标注规范、训练/验证/测试集划分
- 文档图像增强
- 损失函数：MSE、L1/L2....自行选择（或许需要组合）不同的Loss
- 成本控制：服务器预算约 30 元，训练前先用小数据和少量 Epoch 验证流程，正式训练时记录时长与费用

验收标准

- 使用 Git 创建仓库，上传requirement.txt用于记录环境配置
- 输出 Loss 曲线，与PSNR、SSIM演化曲线（例如总共训练1000steps，那么可以每20steps输出一次相比 GT 的 PSNR 和 SSIM 的变化曲线）
- 可视化结果在大部分图片上不出现全白、全黑等情况，输出正确的推理结果
- 思考当前结构的改进方案，如Gan、Diffusion Model